What2It: Sistema di Raccomandazione di Ricette basato sugli Ingredienti Posseduti

Francesco Giovanardi Alessandro Cavina

21 novembre 2025



Sommario

Questo documento descrive l'architettura, le tecnologie e le funzionalità di What2It, un sistema di raccomandazione di ricette che combina disponibilità di ingredienti dell'utente e similarità rispetto alle ricette preferite per proporre piatti pertinenti. Il sistema è realizzato con un frontend web in Streamlit, un database relazionale PostgreSQL e un motore di similarità basato su rappresentazioni TF–IDF e similarità coseno.
Il progetto è open source ed è disponibile pubblicamente su GitHub:
https://github.com/fragiovis/what2it, chiunque può consultare o clonare il codice.

1 Introduzione e Obiettivi del Sistema

L'obiettivo di What2It è aiutare l'utente a rispondere alla domanda "Cosa posso cucinare con quello che ho?". Il sistema:

- consente all'utente di indicare gli ingredienti posseduti;
- mostra ricette adatte, ordinate in base a un punteggio che combina:
  - disponibilità degli ingredienti posseduti;
  - similarità con le ricette preferite in passato.
- permette di salvare/rimuovere ricette preferite e di esplorare ricette simili a quelle già apprezzate.

Il sistema è pensato per un utilizzo immediato via browser, con un'interfaccia semplice e reattiva, e con calcoli di similarità efficienti e riutilizzabili.

2 Tecnologie Utilizzate

2.1 Stack Tecnologico e Motivazioni

Lo stack tecnologico del progetto è basato su:

- **Python**: linguaggio principale per velocità di sviluppo e integrazione DB.
- **Streamlit**: framework web per prototipazione rapida di interfacce dati–centriche. Consente layout a schede, componenti interattivi (pulsanti, liste) e caching integrato per migliorare la reattività.
- **PostgreSQL** (con `psycopg2`): database relazionale robusto e maturo. Viene usato per gestire ricette, ingredienti, relazioni ricetta–ingredienti e preferenze/ingredienti dell'utente.
- **Scikit–learn**: per la rappresentazione testuale TF–IDF del "corpus" di ricette (ingredienti come termini) e il calcolo della similarità coseno tra ricette.
- **NumPy**: supporto alle elaborazioni numeriche e matrici sparse/dense.

2.2 Analisi del Dataset Scelto

Il dataset di partenza (cartella `data/processed/italian gastronomic recipes dataset`) contiene:

- un elenco di ricette con metadati (es. nome, categoria, link, costo, difficoltà, tempo di preparazione);
- un elenco di ingredienti con classi di appartenenza;
- una mappatura ricetta–ingredienti che permette di ricostruire gli ingredienti necessari per ciascuna ricetta.

Provenienza e pulizia dei dati

- Il dataset è stato originariamente scaricato da **Kaggle**.
- È stato **successivamente tradotto in italiano**.
- Sono state **corrette le inconsistenze** presenti, in particolare le mancate corrispondenze tra le ricette e i relativi ingredienti (ricette senza tutti gli ingredienti o ingredienti non referenziati correttamente).

Durante il caricamento (`populate_database.py`) i dati vengono normalizzati nelle tabelle SQL principali:

- `recipes(recipe_id, recipe_name, recipe_link, category_name, category_id, cost, difficulty, preparation_time, image_path)`
- `ingredients(ingredient_id, ingredient_name, ...)`
- `recipe_ingredients(recipe_id, ingredient_id, quantity)`

A supporto delle funzionalità utente:

- `user_owned_ingredients(user_id, ingredient_id)`
- `user_selected_recipes(user_id, recipe_id, selected_at)`

Questa modellazione consente interrogazioni efficienti sia per la disponibilità degli ingredienti sia per le preferenze.

3 Funzionalità del Sistema

3.1 Panoramica

L'applicazione Streamlit espone le seguenti pagine:

- **Login**: gestione utente di sessione.
- **Gestione Ingredienti**: selezione/mantenimento degli ingredienti posseduti (*frigo* dell'utente).
- **In Cerca di Ispirazione**: consigli di ricette filtrate per categoria e riordinate per punteggio combinato (ingredienti posseduti + similarità con preferiti).
- **Le Tue Ricette Preferite**: elenco preferiti con pulsante "Vorrei qualcosa di simile" per suggerire ricette affini.

4 Sistema di Raccomandazione

La logica è implementata nei file `streamlit/pages/In_Cerca_Di_Ispirazione.py`, `streamlit/pages/Le_Tue_Ricette_Preferite.py` e `streamlit/recommendation/compute_item_similarity.py`. Nel seguito analizziamo il flusso che abilita la generazione di raccomandazioni per gli utenti del sistema.

4.1 Costruzione del corpus delle ricette

Si recuperano ricette e ingredienti dal DB e si costruisce un testo per ricetta:

$$\text{testo_ricetta} = \texttt{recipe_name} + \texttt{category_name} + \text{ingredienti concatenati} \tag{1}$$

Si ottiene un **corpus** $\mathcal{C} = [c_1, \ldots, c_N]$ (lista di stringhe) e una lista di mapping `index_to_recipe` $= [(rid_1, name_1), \ldots, (rid_N, name_N)]$. Viene costruita anche la mappa `rid_to_idx` : `recipe_id` $\rightarrow$ indice.

4.2 Matrice TF–IDF

Si applica `TfidfVectorizer` (`lowercase=True`, `ngram_range=(1,2)`) al corpus, ottenendo la matrice term–documento:

$$\mathbf{X} \in \mathbb{R}^{N \times M} \tag{2}$$

dove N è il numero di ricette e M il numero di termini/*n-gram* estratti. $\mathbf{X}$ è tipicamente **sparsa**.

4.3 Matrice di similarità coseno

La similarità tra ricette si calcola come cosine similarity fra vettori TF–IDF, producendo

$$\mathbf{S} = \texttt{cosine_similarity}(\mathbf{X}) \in [0, 1]^{N \times N} \tag{3}$$

$\mathbf{S}$ è simmetrica con diagonale pari a 1. L'elemento S_{ij} misura quanto la ricetta i è simile alla ricetta j in termini di ingredienti e testi associati.

4.4 Cache delle risorse di similarità

Sia in `In_Cerca_Di_Ispirazione.py` sia in `Le_Tue_Ricette_Preferite.py` le risorse vengono **cachate** con `@st.cache_resource`. In particolare si memorizzano:

- **S**: la matrice $N \times N$ di similarità tra ricette;
- `rid_to_idx`: mapping `recipe_id` $\rightarrow$ indice di riga/colonna in **S**;
- (nella pagina dei preferiti) `rid_to_link`: mapping `recipe_id` $\rightarrow$ URL della ricetta.

La cache evita ricalcoli costosi ad ogni interazione, mantenendo reattiva l'interfaccia.

4.5 Generazione delle ricette consigliate (`In_Cerca_Di_Ispirazione.py`)

Per una categoria selezionata, si recuperano con una query SQL le **Top** K ricette ordinate per percentuale di ingredienti posseduti dall'utente:

$$\texttt{owned_ratio}(r) = \frac{\#\,\text{ingredienti posseduti dall'utente in } r}{\#\,\text{ingredienti totali di } r} \tag{4}$$

Per ciascuna ricetta candidata r si calcola anche la **similarità media con i preferiti** dell'utente:

$$\texttt{sim_avg}(r) = \frac{1}{|F|} \sum_{f \in F} S_{\texttt{rid_to_idx}(r),\,\texttt{rid_to_idx}(f)} \tag{5}$$

Infine si ottiene il punteggio finale:

$$\texttt{final_score}(r) = 0.7 \cdot \texttt{owned_ratio}(r) + 0.3 \cdot \texttt{sim_avg}(r) \tag{6}$$

Le ricette vengono ordinate per `final_score` decrescente e presentate all'utente con metadati (categoria, costo, difficoltà, tempo, percentuale posseduta).

4.6 Ricette simili ai preferiti (`Le_Tue_Ricette_Preferite.py`)

Nella pagina dei preferiti, il pulsante "Vorrei qualcosa di simile" usa **S** per selezionare le top 3 ricette simili ad una preferita f, escludendo f stessa. Si sfrutta `rid_to_idx` per indicizzare **S** e si mostrano i risultati con link di dettaglio (se disponibili).

5 Considerazioni di Implementazione e UI

- La UI usa contenitori con bordo e colonne per disporre immagini, titolo e pulsanti (salva/salvato) mantenendo leggibilità.
- Le immagini delle ricette vengono mostrate quando il file `image_path` esiste; si costruisce il percorso assoluto rispetto alla root del progetto.
- Lo stato del pulsante "Salva/Salvato" è coerente con l'appartenenza ai preferiti; le modifiche aggiornano il DB e forzano un rerender.
- La cache delle risorse di similarità è condivisa e ricalcolata solo quando le dipendenze cambiano, riducendo il tempo di risposta.

6 Sviluppi futuri

Il progetto offre numerose possibilità di evoluzione e miglioramento, tra cui:

- L'adozione di un dataset più ricco e aggiornato, anche tramite tecniche di scraping automatico, così da integrare nuovi dati e aumentare la varietà delle raccomandazioni proposte.
- La possibilità per gli utenti di aggiungere nuove ricette e ingredienti direttamente al database, favorendo la crescita collaborativa del patrimonio informativo della piattaforma.
- La trasformazione della piattaforma in un vero e proprio social network culinario, in cui gli utenti possono interagire tra loro, seguirsi a vicenda e scambiarsi suggerimenti sulle ricette. In tale scenario sarà possibile implementare sistemi di raccomandazione basati sull'analisi delle relazioni e delle affinità tra utenti (User-Based Recommendations).
- L'integrazione di un sistema di image detection capace di riconoscere automaticamente i prodotti presenti nel frigorifero dell'utente tramite hardware dedicato, per generare suggerimenti mirati sulla base degli alimenti effettivamente disponibili.